HDOC-59

Longest Unique Substring Using Pointers in C

Problem Statement

Write a C program that reads a string s and passes the string to a function named longest_unique_substring() using its address. Use pointers to pass the string between the calling function and longest_unique_substring(). The function must find the longest substring containing no repeated characters, calculate and display its length, treat uppercase and lowercase letters as different characters, and display 0 if the input string is empty.

Algorithm / Logic

1. Read the input string in main() using fgets() and remove the trailing newline, if present.
2. Pass the address of the string to longest_unique_substring() as a char pointer.
3. If the first character is the null character, display length 0 and return.
4. Use a sliding-window technique. Store the most recent index of each possible unsigned character in an array of 256 elements initialized to -1.
5. Move a pointer through the string. If the current character already occurred inside the current window, move the start of the window to one position after its previous occurrence.
6. Calculate the current unique-substring length and update the best start position and maximum length whenever a longer substring is found.
7. Display the longest unique substring and its length. Uppercase and lowercase characters remain distinct because their character codes are different.

C Program

```
#include <stdio.h>
#include <string.h>

void longest_unique_substring(char *s)
{
    int last[256];
    int start = 0, bestStart = 0, maxLen = 0;
    int i, currentLen;
    unsigned char ch;

    if (*s == '\0') {
        printf("Length of longest substring: 0\n");
        return;
    }

    for (i = 0; i < 256; i++)
        last[i] = -1;

    for (i = 0; *(s + i) != '\0'; i++) {
        ch = (unsigned char)*(s + i);

        if (last[ch] >= start)
            start = last[ch] + 1;

        last[ch] = i;
        currentLen = i - start + 1;

        if (currentLen > maxLen) {
            maxLen = currentLen;
            bestStart = start;
        }
    }

    printf("Longest unique substring: ");
    for (i = bestStart; i < bestStart + maxLen; i++)
        putchar(*(s + i));
```

```
    printf("\nLength of longest substring: %d\n", maxLen);
}

int main(void)
{
    char s[1000];

    printf("Enter a string: ");
    if (fgets(s, sizeof(s), stdin) == NULL)
        s[0] = '\0';

    s[strcspn(s, "\n")] = '\0';
    longest_unique_substring(s);

    return 0;
}
```

Explanation

The function receives the string through a pointer (char *s), so the array address is passed from main() without copying the whole string. The expression *(s + i) accesses characters using pointer arithmetic.

The program uses a sliding window and an array named last[] to remember the latest index of each character. When a repeated character appears inside the current window, the window start is advanced. This avoids checking every possible substring and gives O(n) time complexity for a string of length n, with O(1) extra space because the last[] array has a fixed size of 256.

Character matching is case-sensitive. Therefore, A and a are treated as different characters.

Test Cases

S.No.	Input	Expected Longest Substring	Length
1	abcabcbb	abc	3
2	bbbbb	b	1
3	pwwkew	wke	3
4	AaBbAa	AaBb	4
5	(empty string)	-	0

Sample Output

```
Example 1:
Enter a string: abcabcbb
Longest unique substring: abc
Length of longest substring: 3

Example 2 (empty input):
Enter a string:
Length of longest substring: 0
```

Result

The C program successfully finds and displays the longest substring containing no repeated characters and its length. The string is passed using its address and accessed through pointers. Uppercase and lowercase characters are treated as different, and an empty input correctly produces a length of 0.